UK Train Rides: Analysis, Forecasting and Dashboard

Project Documentation, Deliverable 1: Planning and Management, Literature Review, Requirements Gathering

Team 4

1. Project Planning and Management

1.1 Project Proposal

Overview. The project analyses a UK train ticket-sales dataset (journeys, tickets, prices, delays, refunds) to show rail decision makers what drives demand, revenue and delays, then forecasts next month's rides and revenue. It follows the four DEPI phases: data model and cleaning, analysis questions, forecasting questions, and a Tableau dashboard with a final report and presentation.

Problem statement. Operations, revenue and planning managers need to know which routes, ticket types and times earn revenue, where delays and cancellations drive refunds, and what to expect next month. Without a clean data model and defensible forecasts, these decisions rest on intuition.

Objectives

1. Build a documented star-schema data model and a reproducible cleaning pipeline.
2. Define and answer a broad set of analysis questions with SQL and Python.
3. Answer the three required forecasting questions: number of rides next month, forecast revenue for each day of next month, and demand by ticket class.
4. Make the forecasts defensible with baseline comparison, time-based validation, error metrics and prediction intervals.
5. Deliver one Tableau dashboard that answers every question.
6. Deliver the full DEPI documentation on GitHub, plus the final report and presentation.

Scope.

data modelling and cleaning, descriptive analysis, scikit-learn forecasting, Tableau dashboard, documentation, testing.

1.2 Project Plan

Task	Owner	W1	W2	W3	W4	W5	W6	W7	W8
Documentation	A	■			■			■	■
Dataset download and first look	B	■							
Technical week 1: data model, cleaning, notebook	D, E, A		■						
Technical week 2: analysis questions and answers	D, E			■					
System analysis and design documents	D, A, E		■	■	■				
Technical week 3: forecasting questions and models	C, A, E, B				■	■			
Test plan and test cases	D				■	■	■		
Technical week 4: dashboards	B, C, E					■	■	■	
Code cleanup, README, deployment link	A, E						■	■	
Testing, bug reports, user manual, technical docs	D							■	■
Presentation	All				■	■	■	■	■
Final report	A							■	■

Milestones

Date	Milestone
16 Oct	Items 1 to 3 submitted on GitHub (internal target Thu 15 Oct)
22 Oct	Cleaned dataset and preprocessing notebook committed
29 Oct	Analysis question set with answers committed
5 Nov	System analysis and design documents ready (due 6 Nov)
12 Nov	Three required forecasts, baseline comparison and plots done
26 Nov	Dashboard complete and code frozen
30 Nov	Implementation submitted: code, README, deployment link
3 Dec	Testing report, user manual and presentation dry run done
4 Dec	Final presentation and reports submitted

Deliverables by deadline

- 16 Oct: this document (Planning and Management, Literature Review, Requirements Gathering).
- 6 Nov: system analysis and design set (problem statement, use case diagram, architecture, ERD, schemas, DFD, behavior diagrams, wireframes, UI/UX guidelines, tech stack, deployment and component diagrams), plus cleaned dataset, notebook and analysis question set.
- 30 Nov: source code, version history, README, forecast plots, dashboard and deployment link.
- 4 Dec: test plan, test cases, bug reports, user manual, technical documentation, final report, presentation and optional demo video.

1.3 Task Assignment and Roles

Four owners carry the critical path. One floater absorbs overflow and trivial tasks. Every owner has a named backup.

Member	Role	Owns	Backup
A	Project Lead	Plan, GitHub repo and README, document consolidation, deadline tracking, risk and KPI log, final report, forecasting. Backup: Data	E
B	Visualization Lead	Tableau, Power BI, Looker dashboards. Backup: Forecasting	E, C
C	Forecasting Lead	Forecasting questions, features, models, baseline, validation, forecast plots, Visualizations.	A, E, B
D	Data Lead	SQL data model, ERD and schemas, cleaning and preprocessing notebook, analysis questions and answers	E, A
E	Floater	Data, Forecasting, Visualizations, documentation.	-

1.5 KPIs

Targets are proposals to confirm with the lecturer.

Area	KPI	Proposed target
Delivery	Documents submitted by each DEPI deadline	100% on time
Delivery	Internal milestones met on or before the Thursday date	At least 80%
Data quality	Completeness of key fields after cleaning (price, dates, stations, status)	100%, remaining gaps documented
Analysis	Analysis questions answered with query or code, each tied to a dashboard view	At least 15
Forecasting	Required forecasts delivered	3 of 3
Forecasting	Validation MAE against the naive baseline	Beats the baseline for each forecast
Forecasting	Forecast plots showing MAE, MAPE and intervals	100%
Dashboard	Questions visualised, filters working	100%, date, route and ticket class filters
Dashboard	Load time on Tableau Public	Under 5 seconds
Testing	Test cases passed at the final run, failures logged as bugs	At least 90%
Reproducibility	Fresh clone runs end to end from the README, verified by a member who did not write it	Yes
Team	Pull requests reviewed by another member, commits from every member	100%, every member

2.3 Final Grading Criteria

The DEPI guidelines break marks down by documentation, implementation, testing and presentation. The weights have not been shared yet. [Lecturer to provide the breakdown.]

Area	Marks	Where our evidence lives
Documentation	[TBD]	/docs and the six DEPI documentation milestones
Implementation	[TBD]	/sql, /notebooks, /dashboard, README, commit history
Testing	[TBD]	Test plan, test cases, bug reports
Presentation	[TBD]	Slides

Once the weights are known, the team re-checks its effort split against them.

3.1 Stakeholder Analysis

The dataset is public, so the organization's decision makers are represented by realistic roles.

Stakeholder	Role	Needs from the project	Influence	Interest
Rail operations manager	Runs services	Where and when delays and cancellations happen, and why	High	High
Revenue and pricing manager	Owns ticket pricing	Revenue by ticket class, type, route and time, plus next-month revenue forecast	High	High
Customer service and refunds lead	Handles refunds	Link between delays, cancellations and refund requests	Medium	High
Commercial and marketing manager	Runs promotions and rail cards	Rail card usage, online versus station sales, payment methods	Medium	Medium

Stakeholder	Role	Needs from the project	Influence	Interest
Network and capacity planner	Plans capacity	Peak times, busiest routes, demand by ticket class, next-month rides	Medium	High
Lecturer and DEPI assessors	Assess the project	Complete documentation, working code, clear presentation	High	High
Project team	Builds it	Clear scope and a workable plan	High	High
Passengers (indirect)	Customers	Better reliability and pricing as a downstream effect	Low	Medium

3.3 Functional Requirements

ID	Requirement	Priority
FR-01	Load raw data into a SQL database and keep the raw data unchanged	Must
FR-02	Build a star schema: rides fact table with date, station, ticket and journey status dimensions	Must
FR-03	Clean the data: missing values, data types, duplicates, inconsistent labels	Must
FR-04	Derive fields: delay minutes, hour, day of week, month	Must
FR-05	Validate after cleaning (row counts, ranges, key field completeness) and log the results	Must
FR-06	Export cleaned tables for analysis and Tableau	Must

ID	Requirement	Priority
FR-07	Maintain a list of analysis questions, each with the query or code that answers it	Must
FR-08	Cover revenue, demand, reliability, refunds, rail cards, purchase channel and payment method	Must
FR-09	Produce a Matplotlib chart for each analysis answer	Should
FR-10	Forecast the number of rides for next month	Must
FR-11	Forecast revenue for each day of next month	Must
FR-12	Forecast demand for each ticket class	Must
FR-13	Compare each model with a naive baseline	Must
FR-14	Evaluate with a time-based split or walk-forward validation	Must
FR-15	Report MAE, MAPE and prediction intervals	Must
FR-16	Export forecast tables with date, forecast, lower and upper bound	Must
FR-17	Add candidate improvements from 2.2 after the required forecasts are done	Could
FR-18	One Tableau dashboard visualising the answers to all analysis and forecasting questions	Must
FR-19	Dashboard filters for date, route and ticket class	Must
FR-20	Forecast views show intervals	Must

ID	Requirement	Priority
	and error metrics	
FR-21	Titles and tooltips explaining each chart	Should
FR-22	Publish the dashboard to Tableau Public and share the link	Should
FR-23	Keep all documents, code, notebooks and dashboard files on GitHub with a README	Must
FR-24	Produce the test plan, test cases, bug reports, final report and presentation	Must

3.4 Non-functional Requirements

ID	Category	Requirement
NFR-01	Performance	The main dashboard view loads in under 5 seconds. The cleaning and model pipeline runs in under 15 minutes on a standard laptop
NFR-02	Accuracy	Forecasts beat the naive baseline on validation MAE. Every metric states its validation window
NFR-03	Data integrity	Raw data is never edited. Every cleaning step is scripted and logged. The cleaned data can be rebuilt from raw
NFR-04	Reproducibility	Fixed random seeds, a requirements file with library versions, notebooks that run top to bottom without manual steps
NFR-05	Usability	A first-time viewer finds any answer within three clicks. Color-blind-safe palette, plain-

ID	Category	Requirement
		language titles, units (GBP, minutes) always shown
NFR-06	Security and privacy	No personal data used or published. No credentials in the repository. .gitignore for local files. Dataset licence checked before public sharing
NFR-07	Reliability	Scripts and notebooks handle missing files and bad values with clear errors, not silent failures
NFR-08	Maintainability	Modular, commented code. Consistent naming in Python and SQL. Reusable functions. One purpose per notebook
NFR-09	Portability	Runs on Windows, macOS and Linux with Python 3.10 or newer and the listed libraries. Dashboard opens in any modern browser
NFR-10	Documentation	README covers installation, configuration and execution. Every document is versioned in GitHub with meaningful commit messages